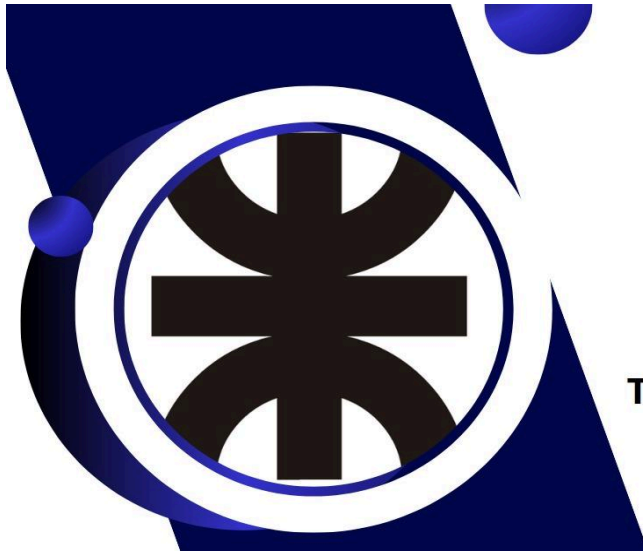




**TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA**

Título del trabajo: Programa de gestión de países

Alumnas:

Chiara Ayelen Aquino, gmail: chiara.aquino01@gmail.com

Yazmin Dana Rodriguez, gmail: yazmindana220@gmail.com

Grupo: 124

Materia: Programación I

Profesores: Cinthia Rigoni, Ariel Enferrel & Martín A. García

Tutores: Franco Gonzalez (Com. 9) & Sofía Fernández (Com. 14)

Fecha de Entrega: 17/06/2026

Índice

Marco Teórico	1
Introducción	1
Conceptos de programación en Python	1
1. Funciones	1
2. Datos primitivos	1
3. F-strings	2
4. Operadores relacionales y lógicos	2
5. Estructuras condicionales	2
6. Estructuras repetitivas	2
7. Listas	2
8. Diccionarios	2
9. Operaciones de entrada y salida	3
10. Manejo de archivos CSV	3
11. Procesamiento de texto	3
12. Herramientas utilizadas	3
Decisiones técnicas y arquitectura del código	4
Arquitectura del código	4
Diagrama de flujo	5
Figura 1.	6
Arquitectura del código mediante capturas de pantalla	7
Figura 2.	7
Figura 3.	7
Figura 4.	8
Dificultades en el desarrollo	9
1. Error de ruta y apertura del archivo CSV	9
2. Codificación del archivo CSV	9
3. Estructura del menú y visualización de resultados	9
Conclusiones	10
Bibliografía	11
Enlace al repositorio del proyecto	12
Enlace al video explicativo del proyecto	12

Marco Teórico

Introducción

El presente marco teórico aborda los conceptos fundamentales de programación en Python necesarios para el desarrollo de un sistema de gestión de información. Se parte de la idea de que un programa estructurado debe organizar sus procesos en unidades lógicas que interactúan entre sí para cumplir un objetivo específico.

Para ello, se describen las estructuras de control, tipos de datos utilizados y técnicas de almacenamiento que permiten manipular grandes volúmenes de información de manera ordenada. Estos conceptos constituyen la base sobre la cual se construye el sistema anteriormente mencionado, ya que define cómo se ingresan, procesan y presentan los datos al usuario.

De esta manera, el marco teórico sienta las bases conceptuales que justifican las decisiones de diseño adoptadas en el desarrollo del trabajo práctico.

Conceptos de programación en Python

1. Funciones

En Python, una función es un bloque de código reutilizable que realiza una tarea específica. Se definen con la palabra clave `def`, seguida del nombre de la función y paréntesis. El uso de funciones permite modularizar el código, es decir, dividirlo en partes más pequeñas y organizadas, donde cada función tiene una única responsabilidad. En este proyecto, cada opción del menú tiene su propia función, lo que facilita la lectura y el mantenimiento del código.

2. Datos primitivos

Python maneja distintos tipos de datos básicos. En este proyecto se utilizaron principalmente:

- `int`: números enteros, usado para población y superficie
- `str`: cadenas de texto, usado para nombre y continente
- `bool`: valores True o False, usado en validaciones

3. F-strings

Los f-strings son una forma de incluir variables dentro de un texto en Python. Se escriben colocando una `f` antes de las comillas y las variables entre llaves `{}`. Por ejemplo:

```
f"País: {nombre}"
```

En este proyecto se usaron para mostrar los datos de cada país de forma clara y ordenada.

4. Operadores relacionales y lógicos

Los operadores relacionales (`==`, `!=`, `>`, `<`, `>=`, `<=`) permiten comparar valores y devuelven `True` o `False`. Los operadores lógicos (`and`, `or`, `not`) combinan varias condiciones. En el proyecto se usaron para validar entradas del usuario y filtrar países según distintos criterios.

5. Estructuras condicionales

Las estructuras condicionales (`if`, `elif`, `else`) permiten ejecutar distintos bloques de código según si una condición es verdadera o falsa. En este proyecto se usaron para manejar las opciones del menú y para validar los datos ingresados por el usuario.

6. Estructuras repetitivas

Python ofrece dos estructuras de repetición principales:

- `for`: recorre una secuencia elemento por elemento. Se usó para iterar sobre la lista de países.
- `while`: repite un bloque mientras una condición sea verdadera. Se implementó en el proyecto para mantener el menú activo hasta que el usuario solicitara salir.

7. Listas

Una lista es una estructura de datos que permite almacenar múltiples elementos en una sola variable, en un orden determinado. Los elementos se pueden agregar con `.append()`, recorrer con `for` y consultar por su posición. En este proyecto, la lista de `países` almacena todos los países cargados desde el CSV.

8. Diccionarios

Un diccionario es una estructura de datos que almacena pares de clave-valor. A diferencia de las listas, los elementos se acceden por su nombre (clave) en vez de por su posición. En

este proyecto, cada país se representa como un diccionario con las claves: nombre, población, superficie y continente.

9. Operaciones de entrada y salida

Estas funciones son fundamentales para la interacción entre el programa y el usuario, y para la lectura de datos desde archivos externos.

- `print( )`: muestra información en pantalla
- `input( )`: permite al usuario ingresar datos desde el teclado
- `open( )`: abre un archivo para leerlo o escribirlo

10. Manejo de archivos CSV

Un archivo CSV (Comma Separated Values) es un archivo de texto donde los datos están separados por comas. Python incluye el módulo `csv` que facilita su lectura y escritura. En este proyecto se usó `csv.DictReader` para leer cada fila del archivo y convertirla automáticamente en un diccionario.

11. Procesamiento de texto

Python ofrece varios métodos para manipular strings:

- `.strip( )`: elimina espacios al inicio y al final
- `.split( )`: divide un texto en partes
- `.lower( )`: convierte a minúsculas, usado para comparaciones sin distinguir mayúsculas

12. Herramientas utilizadas

- `Python 3`: lenguaje de programación utilizado para desarrollar el sistema.
- `VS Code`: editor de código utilizado para escribir y ejecutar el programa.
- `Git y GitHub`: sistema de control de versiones utilizado para gestionar el código y trabajar en equipo de forma colaborativa.

Decisiones técnicas y arquitectura del código

Arquitectura del código

El programa está diseñado con una estructura basada en funciones para dividir la lógica en bloques independientes. Esto facilita el mantenimiento y la reutilización del código, ya que cada función puede modificarse sin afectar al resto del sistema. Se ejecuta dentro de un bucle `while` e implementa un menú que garantiza que el usuario determine cuándo se realiza la lectura del código y cuando se interrumpe.

Al inicio, y como se visualiza en la figura 2, se invoca la función `cargar_paises()` que realiza la lectura del archivo `paises.csv`. Los datos se almacenan en una lista de diccionarios, donde cada diccionario representa un país y sus características: nombre, población, superficie y continente. Se eligió esta estructura porque permite asociar múltiples atributos a cada país y manipular grandes volúmenes de datos de forma ordenada.

Por otro lado, la función `mostrar_menu(paises)` presenta al usuario 8 opciones enumeradas del 0 al 7. Utilizando estructuras condicionales (`if`, `elif`, `else`), se valida la instrucción o solicitud ingresada por el usuario y se ejecuta la función correspondiente según la opción elegida. Esta función se visualiza con mayor detalle en la figura 3 del presente informe.

Durante el desarrollo del proyecto se determinó que el menú debía presentar una opción que permitiera visualizar la lista de países, mostrando por pantalla los registros tanto del CSV como los agregados o modificados por el usuario. La opción número (0), invoca la función `mostrar_paises(paises)`, la cual itera sobre la lista de países utilizando un bucle `for`.

Para finalizar el programa, se estableció como opción número (7) el comando `break`. De esta manera se interrumpe el bucle `while` en el cual se ejecuta el programa y finaliza.

Los cambios realizados no se guardan automáticamente en el archivo CSV. Para que las modificaciones realizadas queden almacenadas, es necesario reescribir el archivo.

Para el control de versiones y trabajo colaborativo se utilizó la plataforma GitHub. El repositorio permitió registrar los cambios realizados, mantener un historial de desarrollo y organizar el código de forma centralizada durante todo el proyecto.

Enlace a repositorio GitHub: <https://github.com/chiaraaquino/TPI---PROGRAMACI-N.git>

Por último, y para finalizar, se realizó un video explicativo del código donde se muestra el funcionamiento del programa, la estructura de las funciones principales y el uso del menú interactivo para la gestión de datos de los países.

Enlace a video explicativo:

https://drive.google.com/file/d/1IBdpzKLEJ1eJBmvfKN_whc55bNWL2wd3/view?usp=sharing

Diagrama de flujo

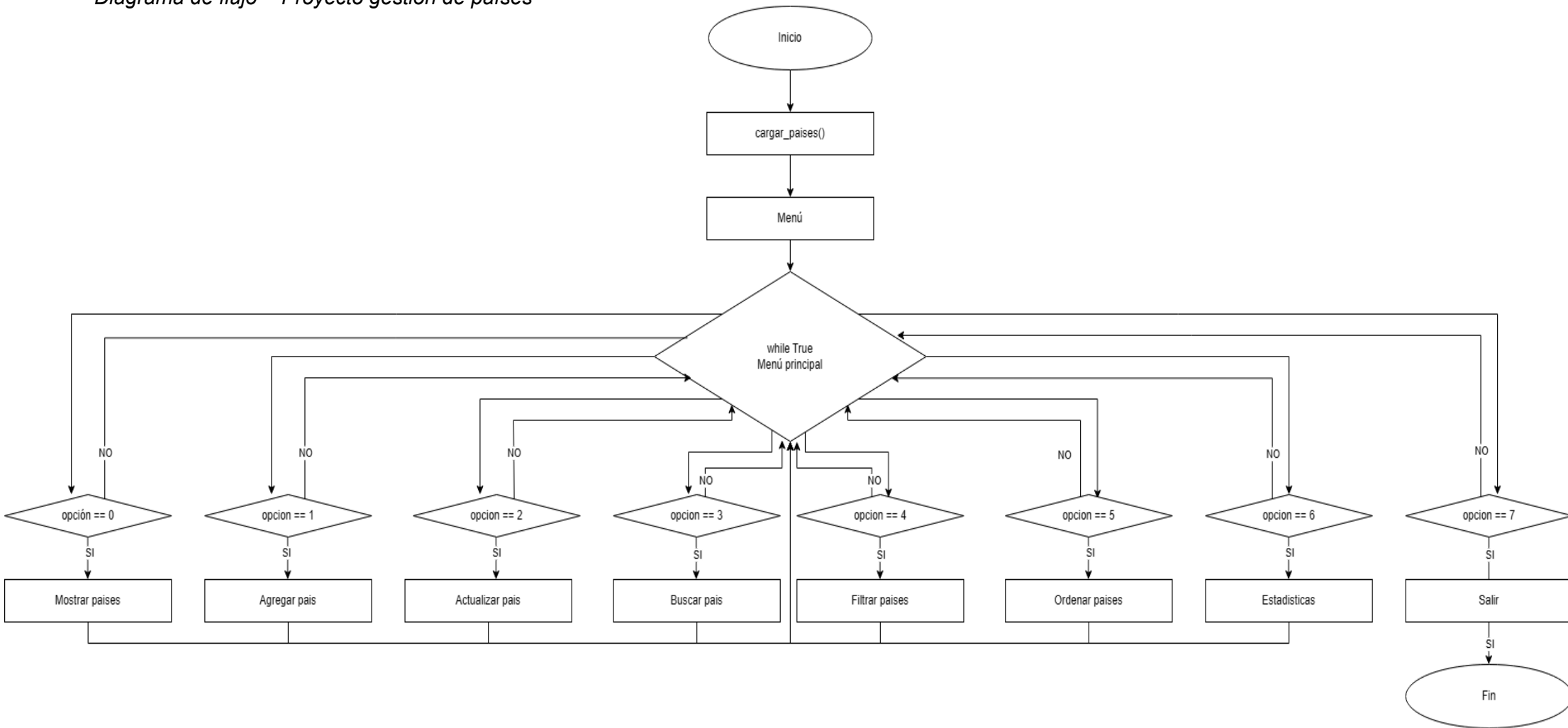
El diagrama de flujo representa la lógica de funcionamiento del programa de gestión de países. La ejecución comienza en un punto de inicio y presenta al usuario un menú con 8 opciones. Luego, mediante una estructura de decisión, el programa evalúa la opción ingresada y deriva el flujo hacia la función correspondiente.

Cada proceso se ejecuta de forma independiente y, al finalizar, el flujo retorna automáticamente al menú principal.

Esto se repite de manera cíclica hasta que el usuario selecciona la opción de salida, momento en el cual el programa finaliza. De esta forma, el diagrama explica de manera visual cómo se organiza la interacción entre el usuario y el sistema, y cómo se mantiene el control del programa dentro de un bucle hasta cumplir el objetivo.

Figura 1.

Diagrama de flujo - Proyecto gestion de países



Nota. El diagrama representa la lógica del menú principal y las siete opciones disponibles: mostrar, agregar, actualizar, buscar, filtrar, ordenar y estadísticas. Cada opción se ejecuta mediante una estructura condicional. Diagrama realizado en draw.io.

Arquitectura del código mediante capturas de pantalla

Figura 2.

Carga de datos desde archivo CSV.

```
1 import csv # importa el módulo csv de Python
2
3 #===== DEFINICIÓN DE FUNCIONES =====#
4 def cargar_paises(): # def : define (o declara) funciones
5     paises = [] # with abre y cierra el archivo automaticamente. # as : nombra a un archivo abierto para usarlo con with
6     ruta = __file__.replace("Programa.py", "paises.csv") # busca el CSV en la misma carpeta que desarrollamos el código.py
7
8     with open(ruta, "r", encoding="utf-8-sig") as archivo: # open() : abre un archivo // r : modo lectura // encoding="utf-8" : permite caracteres especiales como tildes y la ñ.
9         lector = csv.DictReader(archivo, delimiter=',') # csv.DictReader() : convierte cada fila automáticamente en un diccionario
10        for fila in lector:
11            paises.append({ # cada fila se agrega a la lista paises
12                "nombre": fila["nombre"],
13                "poblacion": int(fila["poblacion"]),
14                "superficie": int(fila["superficie"]),
15                "continente": fila["continente"]
16            })
17    return paises # devuelve paises con la info
```

Nota: la función `cargar_paises()` realiza la lectura de `paises.csv` con `csv.DictReader`. Convierte cada fila en un diccionario y lo guarda en una lista.

Figura 3.

Menú interactivo - Proyecto gestion de países

```
238 #===== MENÚ =====#
239 def mostrar_menu(paises):
240     while True:
241         print("\n=== GESTIÓN DE PAÍSES ===")
242         print("0. Mostrar lista de paises")
243         print("1. Agregar país")
244         print("2. Actualizar datos")
245         print("3. Buscar país por nombre")
246         print("4. Filtrar paises")
247         print("5. Ordenar paises")
248         print("6. Mostrar estadísticas")
249         print("7. Salir")
250         opcion = input("Elegí una opción: ")
251
252         if opcion == "0": # MOSTRAR PAISES
253             mostrar_paises(paises)
254
255         elif opcion == "1": # AGREGAR PAÍS
256             agregar_pais(paises)
257
258         elif opcion == "2": # ACTUALIZAR DATOS
259             actualizar_pais(paises)
260
261         elif opcion == "3": # BUSCAR PAÍS POR NOMBRE
262             buscar_pais(paises)
263
264         elif opcion == "4": # FILTRAR PAISES
265             filtrar_paises(paises)
266
267         elif opcion == "5": # ORDENAR PAISES
268             ordenar_paises(paises)
269
270         elif opcion == "6": # ESTADÍSTICAS
271             mostrar_estadisticas(paises)
272             mostrar_estadisticas_continentes(paises)
273
274         elif opcion == "7": # SALIR
275             print("¡Hasta luego!")
276             break
```

Nota: se visualiza en la terminal la salida de las líneas de código 239 a 276, que se ejecutan dentro de un ciclo while.

Figura 4.

Función `mostrar_paises()` - Listado de datos

```
18 #-----OPCION (0) - MOSTRAR PAISES-----#
19 def mostrar_paises(paises):
20     for pais in paises:
21         print(f"Nombre: {pais['nombre']}")
22         print(f"Población: {pais['poblacion']}")
23         print(f"Superficie: {pais['superficie']} km²")
24         print(f"Continente: {pais['continente']}")
25         print("---")
```

Nota: El ciclo `for` recorre la lista de diccionarios e imprime los valores de cada país.

Dificultades en el desarrollo

A continuación, se enumeran algunos obstáculos técnicos presentados durante el desarrollo del proyecto;

1. Error de ruta y apertura del archivo CSV

En la etapa inicial de ejecución, el sistema no reconocía el archivo CSV, generando un error de lectura. Al desarrollar este proyecto en Windows, la ruta de búsqueda definida en el programa no coincidía con la ubicación física del documento, ya que la ejecución podía iniciarse desde directorios distintos al del proyecto.

Para resolverlo, se agruparon el archivo `países.csv` y el script `Programa.py` en una misma carpeta de trabajo. Además, se ajustó la lógica de acceso para que el programa localice el CSV de forma relativa al archivo Python, mediante `ruta = __file__.replace("Programa.py", "países.csv")`.

Esto aseguró que el sistema encuentre el archivo CSV sin depender de la carpeta desde donde se ejecute.

2. Codificación del archivo CSV

Este formato de archivo no realiza correctamente la lectura de países que se escriben con la letra “Ñ” o tildes, logrando que el sistema represente estos caracteres con símbolos que afectan la visualización correcta de los datos.

Para ello, se utilizó el parámetro `encoding="utf-8-sig"`, el cual permite reconocer las tildes y la letra “Ñ”.

3. Estructura del menú y visualización de resultados

Durante el desarrollo del código, se establece el menú con todas las opciones solicitadas. Aunque el formato resultaba efectivo y el código se ejecutaba sin errores, no permitía la visualización adecuada de las solicitudes o actualizaciones realizadas por el usuario.

Debido a esto se implementaron funciones específicas que facilitaron la visualización de las solicitudes o cambios realizados, generando que el usuario logre ver al instante los resultados de sus acciones, confirmando cambios aplicados.

Conclusiones

El desarrollo de este programa demostró que es posible construir una herramienta funcional para la gestión y análisis de información utilizando únicamente las estructuras de Python.

Se logró crear un programa que organiza los datos de manera clara y permite al usuario interactuar de forma sencilla a través de un menú, obteniendo resultados inmediatos según sus necesidades.

Este proyecto refuerza la idea de que la programación no solo resuelve problemas técnicos, sino que también permite modelar situaciones reales y presentar la información de forma ordenada y accesible.

Bibliografía

Python Software Foundation. (s.f). *4. Más herramientas para control de flujo (3.14.6). [4.8. Definir funciones]*. Python. <https://docs.python.org/es/3/tutorial/controlflow.html#defining-functions>

Python Software Foundation. (s.f). *Tipos integrados (3.14.6).*
<https://docs.python.org/es/3/library/stdtypes.html>

Python Software Foundation. (s.f). *7. Entrada y salida. [7.1.1. Formatear cadenas literales]*
<https://docs.python.org/es/3/tutorial/inputoutput.html#formatted-string-literals>

Python Software Foundation. (s.f). *Tipos integrados. [Operaciones booleanas — and, or, not]*
<https://docs.python.org/es/3/library/stdtypes.html#boolean-operations-and-or-not>

Python Software Foundation. (s.f). *4. Más herramientas para control de flujo. [4.1. La sentencia if]*.
<https://docs.python.org/es/3/tutorial/controlflow.html#if-statements>

Python Software Foundation. (s.f). *4. Más herramientas para control de flujo. [4.2. La sentencia for]*.
<https://docs.python.org/es/3/tutorial/controlflow.html#for-statements>

Python Software Foundation. (s.f). *5. Estructuras de datos. [5.1. Más sobre listas]*.
<https://docs.python.org/es/3/tutorial/datastructures.html#more-on-lists>

Python Software Foundation. (s.f). *5. Estructuras de datos. [5.5. Diccionarios]*.
<https://docs.python.org/es/3/tutorial/datastructures.html#dictionaries>

Python Software Foundation. (s.f). *7. Entrada y salida.*
<https://docs.python.org/es/3/tutorial/inputoutput.html>

Python Software Foundation. (s.f). *csv — CSV File Reading and Writing.*
<https://docs.python.org/es/3/library/csv.html>

Python Software Foundation. (s.f). *Tipos integrados*. [Métodos de las cadenas de caracteres].

<https://docs.python.org/es/3/library/stdtypes.html#string-methods>

Microsoft. (2015). *Visual Studio Code* (Versión actual) [Software]. <https://code.visualstudio.com/>

UTN (2019). *Algoritmos y estructuras de programación*

https://frrq.cvg.utn.edu.ar/pluginfile.php/14263/mod_resource/content/0/Algoritmo_PseudoCod.pdf

Enlace al repositorio del proyecto

<https://github.com/chiaraaquino/TPI---PROGRAMACION.git>

Enlace al video explicativo del proyecto

https://drive.google.com/file/d/1IBdpzKLEJ1eJBmvfKN_whc55bNWL2wd3/view?usp=sharing



Video TP Paises.mp4